

category: dotnet-general

tags: [ssh, shell, command-injection, quoting, whitelist, debian, printf, xclip, process-leak]

severity: high

created: 2026-07-26

updated: 2026-07-26

project-origin: RemoteControlTool (CcuClient/CcuUI/ZcuAgent)

Chạy lệnh shell từ .NET (SSH/Process): bộ gotcha quoting — whitelist filename Debian, printf `%`, xclip daemonize

Tình huống gặp phải

Audit bảo mật hệ thống remote-control: nhiều nơi nội suy tham số user vào `bash -c '...'` chạy dưới sudo qua SSH (command injection), ghi crontab bằng `echo`, sync clipboard Linux bằng `xclip`. Fix bằng helper `ShellQuote` (quote + validate whitelist) và gặp 3 gotcha đáng nhớ.

Triệu chứng / Lỗi

1. Whitelist filename alphanumeric+._-+ → cài "pkg_1.0~rc1_amd64.deb" fail oan (tên .deb hợp lệ)
2. Lo ngại printf ăn '%' trong chuỗi user khi ghi crontab
3. xclip: WaitForExit(1000) luôn timeout, Process leak 1 fd mỗi lần sync clipboard

Nguyên nhân gốc rễ (Root Cause)

1. **Version chuẩn Debian chứa `~`** (1.0~rc1 sort thấp hơn 1.0 — convention phổ biến cho RC/beta), và % có thể xuất hiện (URL-encoding). Whitelist viết theo "tên file thông thường" từ chối oan.
2. **`printf '%s\n' '<arg>':** format string là hằng → % trong **argument** là literal, an toàn tuyệt đối. Chỉ nguy hiểm khi đặt chuỗi user vào **chính format string**.
3. **`xclip` cố ý fork/daemonize** để giữ quyền sở hữu X selection (clipboard X11 là pull-model — process phải sống để phục vụ paste) → không bao giờ exit khi còn giữ selection.

Giải pháp

```
// 1. Whitelist filename: cho phép ~ và % ở mọi vị trí TRỪ ký tự đầu
//   (đầu vẫn alphanumeric — chặn tilde-expansion ~user và option-injection -x)
//   Cả hai vô hại trong "$HOME/..." bên trong bash -c '...'

// 2. Ghi crontab an toàn: printf '%s\n' '<nội-dung-đã-single-quote>' | crontab -
//   Xóa job cuối cùng → crontab -r (pipe chuỗi rỗng vào crontab - sẽ fail)
//   Single-quote giữ nguyên $, backtick, newline. Check ExitStatus — đừng nuốt lỗi.

// 3. xclip: ghi stdin async → đóng stdin → Dispose Process NGAY (release handle)
//   KHÔNG WaitForExit, KHÔNG kill process tree (kill = mất clipboard vừa set)
```

Kèm nguyên tắc chung: mọi tham số vào `bash -c` phải qua 1 helper duy nhất (`Quote()` single-quote escape `'\''` + validate whitelist theo loại: filename/username/package) — không escape thủ công rả rác.

Áp dụng lại (How to reuse)

- Viết whitelist filename cho lệnh cài đặt Linux → nhớ `~/%` của Debian ngay từ đầu; test với `pkg_1.0~rc1_amd64.deb`.
- Ghi file cấu hình qua SSH → `printf '%s\n' '<arg>'` (không `echo` — không dịch `\n`, không portable `-e`).
- Gọi tool clipboard/daemon-style (`xclip`, `xsel`) từ .NET → dispose handle ngay, không chờ/kill.
- Grep `bash -c` + string interpolation trong codebase → mỗi chỗ là 1 điểm injection tiềm năng.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ `ProcessStartInfo.ArgumentList` (không phải chuỗi `Arguments`) khi gọi tool local — 1 phần tử = 1 argv, khỏi quoting tay; nhưng với lệnh QUA SSH thì vẫn phải quote vì bên kia là shell.
- ⚠ `Quote()` strip newline có chủ đích khi lệnh SSH phải là 1 dòng — document rõ.
- ⚠ Whitelist đừng nói thêm ký tự shell-active khác (`$`, backtick, `;`, space) — chỉ mở đúng cái cần.

Tham chiếu

- Project liên quan: `IPGS.RemoteControl.CcuClient/ShellQuote.cs` (S1/Q12, commits `0146cb4`, `de981cf`), `CcuUI/Views/CronJobWindow.axaml.cs` (A1/Q13, `58909eb`), `ZcuAgent/Net/ClientSession.cs` (L2/S2, `1ab4f03`)
- GOTCHAS repo RemoteControlTool: G012, G014, G015

Bổ sung 2026-07-26 — `sudo` qua SSH exec channel: phải LUÔN dùng `-S`, đừng bao giờ chạy sudo trần

Bối cảnh: siết bảo mật — chuyển password sudo từ `echo 'pass' | sudo -S` (lộ qua `ps -ef`) sang ghi vào STDIN của SSH channel (`SshCommand.CreateInputStream()`, SSH.NET 2025.1.0).

Lỗi gặp phải: `sudo: no tty present and no askpass program specified`

Nguyên nhân: code chỉ rewrite `sudo` → `sudo -S -p ''` khi có password

(`sudoCount > 0 && password != ""`). Khi password rỗng, nhánh else chạy ``sudo` trần`.

`SshClient.CreateCommand()` dùng SSH *exec channel* — **KHÔNG** cấp `pseudo-tty` — nên sudo trần cố mở `/dev/tty` để hỏi password và thất bại. Đây là lỗi thiết kế **nhánh điều kiện**, không phải lỗi quoting hay thứ tự gọi API.

Bảng chẩn đoán nhanh — 3 thông báo của sudo nói 3 chuyện khác nhau:

Thông báo sudo	Ý nghĩa thật
---	---
<code>no tty present and no askpass program specified</code>	Chạy <code>sudo</code> thiếu <code>-S</code> trên kênh không có tty
<code>no password was provided</code>	Có <code>-S</code> nhưng stdin EOF/rỗng
<code>3 incorrect password attempts</code>	Có <code>-S</code> , có dữ liệu, nhưng password sai

Quy tắc rút ra:

1. Hễ lệnh có `sudo` là **LUÔN** thêm `-S -p ''`, không phụ thuộc có password hay không.
2. Thiếu password → báo lỗi **ngay tại phía client**, đừng đẩy sang máy remote nhận thông báo khó hiểu.
3. Log lệnh thực sự gửi đi — an toàn vì password chỉ đi qua stdin, không nằm trong command line.

`CreateInputStream()` — thứ tự đúng (đã kiểm chứng):

Phải gọi **SAU** khi `ExecuteAsync()` đã bắt đầu. Chuỗi lỗi trong `Renci.SshNet.dll` là

"The input stream can be used only during execution", và doc chính thức dùng đúng thứ tự này:

```
var execTask = cmd.ExecuteAsync();  
using (var input = cmd.CreateInputStream()) { input.Write(passBytes); }  
await execTask;
```

Nhưng `ExecuteAsync()` mở channel **bất đồng bộ** → nên retry ngắn khi `CreateInputStream()`

ném `InvalidOperationException`, thay vì để hỏng cả lệnh.

KHÔNG cần làm lại:

- Đảo `CreateInputStream()` lên **trước** `ExecuteAsync()` — sai, chắc chắn ném exception.
- Cấp pseudo-tty (`ShellStream`) để chạy sudo — `sudo -S` đọc stdin là đủ; thêm tty còn làm password bị echo ngược vào output.
- Quay lại `echo 'pass' | sudo -S` — làm lộ password qua `ps -ef` trên máy remote.

Bài học quy trình: thay đổi này build sạch 0 error nhưng **hỏng hoàn toàn khi chạy thật** — vì không có môi trường test. Với thay đổi đúng cơ chế xác thực/quyền, build sạch KHÔNG phải bằng chứng hoạt động; phải smoke test trên máy thật trước khi coi là xong.